

goxpyriment — Timing Tests

christophe@pallier.org

2026-07-28

Contents

Timing-Tests: A Guide for Researchers	1
Why timing matters	2
Read this before you trust any number	2
The six tests	2
Recording system information (-sysinfo)	3
Understanding the display refresh cycle	3
No hardware needed	4
Photodiode and/or trigger box	5
Interpreting results: what is “good”?	9
Loading data in Python	9
Improving timing on your system	10
Audio buffer size	11
Walkthrough: a Raspberry Pi, GPIO triggers, and BBTK capture	12
Related tests	14
DLP-IO8	15
Parallel port and GPIO alternatives	16

Timing-Tests: A Guide for Researchers

This document explains how to use the Timing-Tests program to characterise the timing behaviour of your computer before running a psychophysical experiment.

Quick reference — flags, equipment, and one-line test descriptions are in `tests/Timing-Tests/README.md`. This document explains the *why*.

For the EEG/MEG question specifically — how consistent the delay is between a TTL trigger and the physical stimulus — see [Minimising trigger-to-stimulus jitter](#), which gives the measured decomposition and the two things that matter most.

Why timing matters

In a psychology experiment every stimulus has an intended onset and offset time. Whether you are presenting a word for exactly 100 ms, playing a tone that should coincide with a visual flash, or measuring a reaction time to the nearest millisecond, you are trusting the computer to do two things correctly:

1. **Present stimuli when you ask it to.** If you ask for a 100 ms word, does the word actually appear on screen for 100 ms?
2. **Record timestamps accurately.** If you record the time of a key press, is that the time the key was physically depressed, or is it delayed by polling?

Neither is guaranteed. Both depend on your operating system, graphics driver, audio driver, and hardware. This suite lets you measure them on *your specific machine*, so you can report them in your methods section and fix problems before data collection begins.

Read this before you trust any number

The statistics these tests print come from software timestamps. They record when goxpyriment *believes* a flip happened — not what reached the panel.

This is not a theoretical caveat. In July 2026 a presentation bug on a GNOME/Wayland machine left the display showing stale frames for *seconds at a time*, while the program's own numbers stayed textbook-perfect throughout: bright-phase duration 199.915 ± 0.16 ms against a 200 ms target, cycle period 500.05 ± 2.45 ms. Nothing in the console output hinted at a problem. It was visible only with a photodiode. (Cause and fix are documented in `apparatus/CLAUDE.md`; the regression test is `tests/test_clear_only_frames`.)

So:

- **Software statistics tell you about the software.** They are genuinely useful for detecting dropped frames, scheduler jitter, and audio-buffer effects.
- **A photodiode and a trigger box tell you about the experiment.** Only they can confirm that light actually changed when you said it would.

If you are validating a rig for publication, measure it with hardware. The av test is built for exactly that.

The six tests

Two groups: what runs on any machine, and what needs equipment.

No hardware needed

- check Go/no-go: can this machine display and make noise at all?
- display What is my true refresh rate, and how stable are frame intervals?
- latency How long does SDL hold audio before it sounds?

Photodiode and/or trigger box

av THE stimulus timing test – visual + audio + TTL, every cycle
vrr Can I present arbitrary durations, not just multiples of a frame?
rt How precise is my reaction-time measurement?

Two tests that used to live here now have their own directories: tests/test_gv_sync (.gv playback synchronisation) and tests/test_dlpio8 (DLP-IO8-G square-wave characterisation).

This document assumes you have built the program:

```
go build -o Timing-Tests ./tests/Timing-Tests
```

Recording system information (-sysinfo)

Before running anything, capture a snapshot of the machine. -sysinfo prints it and exits — no window, no SDL:

```
Timing-Tests -sysinfo
```

```
Machine:    product: Precision 5490  System: Dell Inc.  Type: laptop
System:     Host: mylab-pc  Kernel: 7.0.0-28-generic x86_64  Uptime: 3h 12m
            OS: Ubuntu 26.04  Shell: bash 5.2.37  Desktop: ubuntu:GNOME
CPU:        Model: Intel(R) Core(TM) Ultra 7 165H  Info: 16 cores / 22 threads
Memory:     RAM: total: 30.04 GiB  used: 6.21 GiB (20.7%)
Graphics:   Card: Intel Corporation Meteor Lake-P [Intel Arc Graphics]  Driver: i915
Audio:      Server: PipeWire  v: 1.4.7
```

Keep it alongside your results:

```
Timing-Tests -sysinfo > sysinfo-$(hostname)-$(date +%Y%m%d).txt
```

Reviewers routinely ask for CPU model, OS version, graphics driver, and audio server. Every data file also carries this in its -info.txt header, including the **display the window actually opened on** — read it from there rather than assuming, especially on a multi-monitor machine.

Understanding the display refresh cycle

Most monitors refresh at a fixed rate — typically 60 Hz (a new frame every 16.67 ms), 120 Hz, or 144 Hz. Your graphics driver synchronises presentation to this cycle (VSYNC). Two consequences:

- **Durations are quantised to multiples of the frame period.** At 60 Hz you can show a stimulus for 16.67, 33.33, or 50 ms — but not 25 ms. Plan accordingly, or see the vrr test.
- **Onset is not when you call the flip; it is the next VSYNC**, anywhere from 0 to 16.67 ms later. goxpyriment keeps that offset constant once the pipeline is

warm, but the first frames should be excluded — `-warmup` does this.

And the screen does not change all at once

An LCD paints top to bottom. On the rig this suite was developed against, a square at the bottom of the screen lights **13.15 ms** after one at the top — close to a full frame period. Measured on a 1280×1024 @ 60.02 Hz panel, that is **15.96 μ s per pixel row**.

This matters more than it sounds. A stimulus at screen centre appears ~6.6 ms after one at the top. If your photodiode sits in a corner but your stimulus is central, your measured onset is biased by that amount — a systematic error larger than most of the jitter people worry about.

The `av` test draws five squares (four corners plus centre) precisely so you can measure this on your own display. Put a photodiode on a top square and another on a bottom one; the difference is your panel's scan-out gradient. Two squares on the same row are only microseconds apart and serve as a sanity check. Each display needs its own figure.

No hardware needed

check — does anything work at all

```
Timing-Tests -test check
```

Flashes a white screen for a second, then plays two sounds. If you do not see the flash or hear both sounds, stop and fix that first. Common causes: the window opened on the wrong monitor (`-d`), the audio device is muted, or the default output is not your speakers.

Measures nothing. It exists so you do not waste a session discovering the hardware was not plugged in.

display — true refresh rate and frame stability

```
Timing-Tests -test display -duration-s 30
```

Flips a uniform screen for `-duration-s` and reports the distribution of frame-to-frame intervals, plus the refresh rate implied by their mean. Use that measured rate for `-hz` in later tests rather than assuming 60.

What to look for: a tight distribution centred on your nominal frame period. A long right tail means dropped frames — a compositor, a background process, or power management. A *bimodal* distribution usually means the display is not running at the rate it reports.

latency — audio pipeline delay

```
Timing-Tests -test latency
Timing-Tests -test latency -audio-frames 256 -drain-reps 20
```

Plays tones of known duration and spin-polls until the audio stream has drained, measuring how long SDL holds PCM beyond the nominal duration. That difference is your audio output latency.

No hardware needed — but note it measures the *software* pipeline, not speaker-to-microphone delay. For true acoustic onset, use av with a microphone.

Photodiode and/or trigger box

av — the stimulus timing test

This is the one that matters. Every cycle presents all three modalities at once:

- **Visual** — five squares (four corners + centre) bright for -frames-on frames, then dark for -frames-off frames
- **Audio** — a tone lasting frames-on × frame-period, synchronised to the visual onset or offset from it by -soa-ms
- **TTL** — a -trigger-ms pulse at the visual onset, on the device chosen by -trigger-device (see below)

```
# 200 ms on, 300 ms off at 60 Hz, 100 cycles – the defaults
```

```
Timing-Tests -test av -cycles 100
```

```
# Visual only, no hardware at all
```

```
Timing-Tests -test av -no-sound -no-ttl
```

```
# Audio leading the flash by 50 ms
```

```
Timing-Tests -test av -soa-ms -50
```

-no-sound and -no-ttl drop a modality. This is how one test covers what used to be three: -no-sound is a pure visual-onset test, and the audio channel of a recording gives tone-onset jitter over a long session.

What to record. A photodiode on one square, the TTL line on a second channel, a microphone if you are checking audio. The quantity of interest is the *offset* between the TTL edge and the luminance step, and how stable it is across cycles. A constant offset is a calibration you subtract; a varying one is jitter you cannot.

What the console tells you. Bright-phase duration, cycle period, and (with sound) audio-queued minus visual onset. All software-side — cross-checks, not ground truth. See the warning at the top of this document.

Reference numbers. Dell Precision 5490, external 1280×1024 @ 60.02 Hz display under Wayland: 100/100 cycles clean, TTL→photodiode 29.878 ± 0.628 ms (from cycle 10 on), top-to-bottom gradient 13.150 ± 0.221 ms, ~5 dropped frames per 100

cycles. Same machine on a bare console with `SDL_VIDEODRIVER=kmsdrm`: 58/58 cycles, TTL→photodiode 21.51 ± 1.19 ms, **zero** dropped frames. Compositing costs roughly one extra frame of latency and a few percent of dropped frames.

Expect a warm-up transient. The first six cycles on that rig ran 36–39 ms before settling to ~ 27 ms. `-warmup 10` excludes them from the test’s own statistics, but offline analysis tools do not know about it — quote steady-state numbers, and say which cycles you used.

Audio onsets are quantised by the buffer. With `-audio-frames 256` at 44100 Hz, tone onsets land on 5.8 ms steps. This appears as a lattice in the inter-onset intervals and is not jitter — it is the buffer. Halving the buffer halves the step, at the cost of underrun risk.

Choosing a TTL output device

`-trigger-device` selects among five back-ends. They are **not** interchangeable. The trigger is fired immediately after the flip returns, so whatever the write costs falls between the flip and the TTL edge — and it shows up in the trial-to-trial *spread* of that interval, not as a constant offset you could subtract afterwards.

-trigger-device	Path to the pin	Logic	Extra flags
dlpio8 (default)	USB serial (FTDI)	5 V	-port
parallel	ioctl on ppdev	5 V	-parallel-port
gpio	ioctl on a GPIO chip	3.3 V	-gpio-chip, -gpio-pins
ft232h	USB bulk transfer (MPSSE via usbfs)	3.3 V	— (first device found)
labjackt4	Modbus TCP over the network	3.3 V	-labjack-host (required)

`parallel` and `gpio` are both a local `ioctl` and carry none of the USB frame scheduling that governs the DLP-IO8’s output latency (see the DLP-IO8 section below — the FTDI latency timer does *not* fix that). Prefer `parallel` on a desktop that still has an LPT port, and `gpio` on a single-board computer. `ft232h` and `labjackt4` are for a rig that has neither: both put a link back between the flip and the edge — USB for the FT232H, an Ethernet round trip for the T4 — so they are a fallback, not an improvement on the two `ioctl` paths.

How much each costs here has **not been measured**; the ordering above is what the transport implies, not a result. The TTL channel of the recording is the only thing that settles it for a given machine.

```
Timing-Tests -test av -trigger-device parallel # LPT, auto-detect
Timing-Tests -test av -trigger-device parallel -parallel-port /dev/parport1
Timing-Tests -test av -trigger-device gpio # /dev/gpiochip0, de
Timing-Tests -test av -trigger-device gpio -gpio-chip /dev/gpiochip4 -trigger-pin 2
Timing-Tests -test av -trigger-device ft232h # first FT232H on th
Timing-Tests -test av -trigger-device labjackt4 -labjack-host 192.168.1.100
```

-trigger-pin is 1-8 on all five, but it names a different thing on each. Read what the program prints at start-up and probe *that* pin:

- **dlpio8** — the number printed on the terminal block.
- **parallel** — a data line: pin 1 is D0, which is **DB25 pin 2** (D0-D7 are DB25 pins 2-9). Ground is any of DB25 pins 18-25.
- **gpio** — a *position in -gpio-pins*, not a line number. With the default list 17,27,22,5,6,13,19,26, pin 1 is **BCM 17**.
- **ft232h** — a D-bus line counted from 0: pin 1 is **AD0**, pin 8 is AD7. The C-bus (AC0-AC7) is the input side and is not driven here.
- **labjackt4** — a position in DIO4-DIO11: pin 1 is **DIO4**, the screw terminal marked **FIO4**; pin 8 is DIO11 = EIO3 on the DB15. DIO0-DIO3 are the analog inputs AIN0-AIN3 and cannot be driven, which is why the group starts at DIO4.

Prerequisites on Linux: `parallel` needs `sudo modprobe ppdev` and membership of the `lp` group; `gpio` needs kernel ≥ 5.10 and membership of the `gpio` group; `ft232h` is Linux-only and needs `ftdi_sio` unloaded (`sudo rmmod ftdi_sio`) plus `rw` access to `/dev/bus/usb/BBB/DDD` (udev rule or the `plugdev` group); `labjackt4` needs the T4 reachable on TCP port 502. `usermod` changes need a re-login. Verify the wiring with `tests/test_parallel_port`, `tests/test_linuxgpio`, `tests/test_ft232h` or `tests/test_labjackt4` before spending a long capture.

Whichever device is used is written into the results header as `trigger=...`, because they do not yield the same onset-vs-TTL figure. A session that does not record it cannot be compared with one that does.

A device that fails to open does not stop the run. It logs a warning, disables triggers, and continues — so the visual measurement is still obtained. The trap is that the run then exits *successfully* with no TTL in the trace. Under `BBTK_CAPTURE=1` that spends a capture window on a recording with an empty TTL channel. Watch the start-up lines.

Recording it automatically

`tests/Timing-Tests/run-timing-tests.sh` can drive `bbtk-capture` so the stimulus always falls inside the capture window, without two terminals and a stopwatch:

```
BBTK_CAPTURE=1 BBTK_CAPTURE_BIN=~/.git/bbtkv3/_build/bbtk-capture \
./run-timing-tests.sh av-visual
```

It launches the capture, blocks until the device is *actually* recording, runs the stimulus, then waits for the capture to download and save. Only the photodiode steps are wrapped. `BBTK_MARGIN_S` (default 8) sets how much is recorded either side of the stimulus.

The waiting matters: `bbtk-capture` needs 11-40 s between launch and the device recording — fixed command pacing plus an internal-memory erase whose duration depends on whether the box needs a full format — and its own “Capturing events...” message appears several seconds *before* that instant. The script synchronises on the

BBTK-CAPTURE-READY line emitted the moment the device is armed. Budget that startup for every recorded step.

Microphone coupling

If you are measuring audio, **max the output volume and place the BBTK microphone within a few millimetres of the speaker membrane.** Anything less and events are silently dropped: there is no warning, and the run looks like it worked.

Two checks on the resulting `-events.csv`:

- **N(Mic1) should equal N(TTLin1).** Far fewer means bad coupling, not bad timing — a 197-cycle run yielding 6 Mic events is a placement problem.
- **Mic duration should be close to frames-on × frame period** (200 ms at the defaults). A 20 ms Mic pulse for a 200 ms tone means the threshold is catching only the loudest fraction of the tone.

vrr — arbitrary stimulus durations

```
Timing-Tests -test vrr -vrr-max-ms 50 -cycles 5
```

Why fixed refresh is a problem On a 60 Hz monitor every duration is a multiple of 16.67 ms. You can show a word for 16.67, 33.33, or 50 ms, but not 20 or 25 ms. At 120 Hz the quantum shrinks to 8.33 ms — better, but still coarse for subliminal work where you may want 10, 12, or 15 ms.

Variable Refresh Rate — AMD FreeSync, NVIDIA G-Sync, or the underlying VESA Adaptive-Sync standard — removes the quantisation. The display holds the current frame for exactly as long as you ask, then refreshes when told. Durations become controlled by your software timer rather than the display clock.

How the test works goxpyriment normally presents with VSync enabled, blocking until the next fixed edge. The `vrr` test switches the renderer to `vsync=0` for the duration of the test (restored on exit), then sweeps target durations from 1 ms to `-vrr-max-ms` in 1 ms steps, holding each with a busy-wait and recording what was achieved.

Reading the result

- **On a working VRR display:** duration error stays small and roughly constant across the whole sweep.
- **On a fixed-refresh display:** the error is *periodic* — near zero at multiples of the frame period, rising in between, because the panel can only change on its own schedule. A sawtooth in the per-step output means you do not have VRR, whatever the monitor's box claimed.

The test prints one line per target plus an aggregate over the whole sweep.

```
# Does the connector support it? Look for "vrr_capable: 1"
sudo cat /sys/kernel/debug/dri/0/*/vrr_capable
```

Enabling VRR on Linux Confirm with the test itself rather than trusting the setting — that is the point of having it.

rt — reaction-time timestamp precision

```
Timing-Tests -test rt -cycles 50
```

Flashes the screen at a jittered interval and waits for a key press, recording the SDL3 hardware event timestamp against the flip timestamp. This characterises the *input* path: how much delay and jitter your keyboard, USB stack, and event queue add.

Press as fast as you can, or better, use a solenoid — you are characterising the machine, and human variability swamps it. The useful number is the SD, not the mean.

Interpreting results: what is “good”?

Metric	Excellent	Acceptable	Problematic
Frame-interval SD (display)	< 0.1 ms	< 0.5 ms	> 1 ms
Frames > 0.5 ms late (display)	< 1 %	< 5 %	> 10 %
Audio pipeline latency (latency)	< 15 ms	< 30 ms	> 50 ms
Bright-phase duration SD (av)	< 0.2 ms	< 1 ms	> 2 ms
TTL→photodiode SD (av, hardware)	< 0.5 ms	< 2 ms	> 5 ms
Dropped frames per 100 cycles (av)	0	< 5	> 10
VRR duration error SD (vrr)	< 0.1 ms	< 0.5 ms	> 1 ms (or periodic → no VRR)
RT SD (rt)	< 3 ms	< 10 ms	> 20 ms

The hardware rows are the ones worth quoting in a methods section.

Loading data in Python

Each run writes two files to ~/goxpy_data/, or to -outdir if given (run-timing-tests.sh points it at the session directory so everything from one session stays together):

- Timing-Tests_sub-000_date-<YYYYMMDD>-<HHMMSS>.csv — data rows, no comments
- ...-info.txt — session metadata: start/end time, hostname, OS, and the display and audio configuration actually in use

```
import pandas as pd

df = pd.read_csv("~/goxpy_data/Timing-Tests_sub-000_date-20260728-091541.csv")

# av: drop the warm-up cycles before quoting anything
steady = df[df.cycle >= 10]
print(steady.bright_duration_ms.describe())
print(steady.period_ms.std())
```

The av test writes one row per cycle: cycle, t_visual_before_ms, t_visual_after_ms, bright_duration_ms, period_ms, t_audio_queued_ms, soa_intended_ms, soa_actual_ms.

Always read display parameters from the run's own -info.txt. On a multi-monitor machine the displays differ in resolution, refresh rate and pixel density, and the scan-out calibration is per-display. The header records which one the window actually opened on.

Improving timing on your system

Linux

- **Disable the compositor.** By far the largest single improvement. Start from a virtual terminal (Ctrl+Alt+F2) with the window system stopped (systemctl stop gdm) and SDL_VIDEODRIVER=kmsdrm. On the reference rig this took dropped frames from ~5 % to zero and removed a frame of latency. Failing that, use a plain window manager (i3, openbox).
- Disable CPU frequency scaling: cpupower frequency-set -g performance.
- **Run with real-time scheduling.** Grant your user the privilege once (below); after that, goxpyriment programs — Timing-Tests included — request priority 50 at startup on their own, so nothing needs prefixing:

```
Timing-Tests -test display -duration-s 30      # asks for real-time itself
Timing-Tests -no-realtime -test display        # opt out
chrt -f 50 some-other-program                 # for anything that does not
```

This matters most when a program is launched by clicking its icon, where there is no command line to prefix. If the grant is missing the program says so and continues at normal priority rather than refusing to run, and the Sched: line in its system report records which it got. The privilege comes from a file you add to /etc/security/limits.d/ — **not** from editing /etc/security/limits.conf, which is package-managed and can be replaced on upgrade, taking your setting with it. See [Setting priority under Linux](#) for the procedure: create a group, drop

in `/etc/security/limits.d/99-goxpyriment.conf`, add yourself, log out and back in.

Two traps worth knowing before you spend a session on it:

- **The limit is inherited from your login session.** A new terminal will not pick up the change; only a full logout and login will. Check `ulimit -r` before trusting a run — if it still says 0, the grant is not in effect and `chrt` will simply fail.
- **Editing the file without sudo fails silently in some editors.** Confirm the text actually landed, with `grep rtprio /etc/security/limits.d/*.conf`, rather than assuming the save worked.

Do not reuse an existing group such as `audio` for this. Its `rtprio` grant belongs to another package (`jackd`), which can revoke it on reconfigure — and a run that quietly loses real-time priority looks exactly like a run that had it.

The priority must not exceed the `rtprio` value granted in the setup above. Asking for more fails with “Operation not permitted” – the same error as having no grant at all, so it is easily misdiagnosed. The setup document grants 50, hence `-f 50` here; keep the two numbers equal.

You *can* skip the setup with `sudo chrt -f 50 Timing-Tests ...`, but the test then runs as root: data files land owned by root and it picks up root’s environment, not yours. Fine for a one-off check, wrong for collecting data.

- Consider a real-time kernel — see <https://ubuntu.com/blog/enable-real-time-ubuntu>.
- **Lower the FTDI latency timer** if you read responses over a USB serial device — see the DLP-IO8 section below. Note it does *not* reduce the latency of *sending* a trigger; that is USB frame scheduling, which the timer does not touch.
- **Send triggers off the USB bus if you can.** Since the timer above cannot help with output, the fix is a different device: `-trigger-device parallel` on a machine with an LPT port, or `-trigger-device gpio` on a single-board computer. Both write via one `ioctl`. See [Choosing a TTL output device](#).

Windows

- Disable “Hardware-Accelerated GPU Scheduling” in Display Settings if you see high frame jitter.

Audio buffer size

The hardware audio buffer is controlled by the SDL hint `SDL_AUDIO_DEVICE_SAMPLE_FRAMES`, exposed as `-audio-frames`. It must be set before the audio device opens.

```
# Default (platform-dependent, often 512–2048 frames)
```

```
Timing-Tests -test latency
```

```
# Aggressive low-latency (~5.8 ms at 44100 Hz)
```

```
Timing-Tests -test latency -audio-frames 256 -drain-reps 20
```

```
# Conservative (~46 ms, stable anywhere)
```

```
Timing-Tests -test latency -audio-frames 2048
```

On startup the program prints the actual device format:

```
audio: 44100 Hz 2 ch 256 sample frames (~5.8 ms latency)
```

The buffer sets the floor on audio-onset precision: onsets can only land on buffer boundaries, so 256 frames at 44100 Hz quantises them to 5.8 ms steps. Use latency at several sizes to find the smallest that drains stably (low SD), then use that everywhere — including in your actual experiment.

Walkthrough: a Raspberry Pi, GPIO triggers, and BBTK capture

A full run-`timing-tests.sh` session on a Raspberry Pi, driving the TTL from the GPIO header instead of a USB trigger box and recording it on a Black Box ToolKit. The GPIO path matters here: it removes the USB link from between the flip and the TTL edge, so what remains in the onset-vs-TTL spread is the Pi's display path rather than a serial transaction.

None of the steps below have been run on a Pi by the author of this section. The procedure follows from the flags and the wiring; the numbers it produces are what the session is *for*. Treat any figure quoted from another machine in this document as inapplicable until you have measured this one.

1. Find the GPIO chip that owns the 40-pin header

Do not assume `/dev/gpiochip0`. Which chip carries the header varies by Pi model and kernel — on some combinations the header is *not* chip 0, and an unrelated chip is. Ask the system:

```
sudo apt install gpiod          # once
gpiodetect                     # lists chips, line counts, and labels
gpioinfo | less                # per-line names; the header lines are named GPIOnn
```

Pick the chip whose lines are the header's, and pass it as `GPIO_CHIP`. Getting this wrong is not merely inert — another chip's lines may drive real board hardware.

2. Grant access and verify the wiring

```
sudo usermod -aG gpio $USER    # then log out and back in
go run ./tests/test_linuxgpio -chip /dev/gpiochip0 -out 17,27,22,5,6,13,19,26
```

Confirm on a scope or logic analyser that the pin you intend to use actually toggles **before** going any further. `test_linuxgpio` exists precisely so that a wiring fault is found here rather than in a 1000-cycle capture.

3. Wire the TTL and the photodiode

- **TTL** — the header pin for the line you chose, into a BBTK TTL input. With the default GPIO_PINS, TRIGGER_PIN=1 is **BCM 17** (physical pin 11). Ground to any header ground pin. Timing-Tests prints the BCM number it resolved to at start-up — probe that one, not the flag value.
- **Photodiode** — on the **top-left** stimulus square, at the square's centre. The panel scans top-to-bottom, so the bottom squares light nearly a frame later; a second diode on a bottom square measures that gradient.

3.3 V is not TTL. Pi GPIO swings 0–3.3 V. Whether a given BBTK input latches at 3.3 V is a property of that unit — check it with a 20-cycle run and confirm `N(TTLin1)` equals the cycle count before committing to 1000 cycles. If it does not latch, a 3.3 V→5 V level shifter on the trigger line is the fix; do not feed 5 V back into a Pi input.

4. Get off the desktop

This is worth more than any flag in this document. On the reference hardware the compositor accounted for essentially all of the measured jitter. Run from a bare console with no desktop session:

```
sudo systemctl set-default multi-user.target && sudo reboot # revert with graphical.target
```

Then, in the console, use the KMS/DRM video driver:

```
export SDL_VIDEODRIVER=kmsdrm
```

Confirm the refresh rate the Pi is actually driving before trusting any duration — -frames-on 12 is 200 ms only at 60 Hz:

```
./Timing-Tests -test display -duration-s 30
```

5. Run the session

```
cd tests/Timing-Tests
go build .

TRIGGER_DEVICE=gpio \
GPIO_CHIP=/dev/gpiochip0 \
GPIO_PINS=17,27,22,5,6,13,19,26 \
TRIGGER_PIN=1 \
REFRESH_HZ=60 \
BBTK_CAPTURE=1 \
BBTK_CAPTURE_BIN=~/.git/bbtkv3/_build/bbtk-capture \
./run-timing-tests.sh
```

Check the banner before answering the first prompt — it echoes the trigger device, chip and pins, the tone frequency, and the presented/analysed cycle split. If it says `dlpio8`, the environment variable did not take.

Set BBTK_PORT if the capture tool cannot find the box on its own. Budget 11–40 s of device setup per recorded step, and remember every av step is $\sim 8\frac{1}{2}$ minutes of stimulus at the defaults.

To pilot the wiring cheaply first, shorten the run rather than skipping it:

```
CYCLES=30 WARMUP=5 TRIGGER_DEVICE=gpio ./run-timing-tests.sh av
```

6. Check the capture before believing it

On the resulting `-events.csv`:

- **N(TTLin1) should equal the number of cycles presented** (CYCLES, including the warm-up ones — they fire the TTL too). Fewer means the 3.3 V swing is not latching, not that triggers were dropped.
- **N(Opto1) should equal N(TTLin1)**. A shortfall is photodiode placement or threshold, not timing.
- **Opto duration $\approx$ 200 ms** at the defaults. Note that BBTK smoothing adds roughly 20 ms to raw durations — read `CorrectedDuration`, and do not interpret the raw figure as panel behaviour.
- **The results header should read `trigger=gpio chip=... line=17 ...`**. If it says `trigger=none`, the chip failed to open and the run continued without triggers; the capture's TTL channel is empty and the step must be re-run.

7. What the Pi will not fix

A previous Pi session measured a TTL→Opto lag of **38.5 ms**, against the 4–7 ms range Bridges et al. (2020) report across packages. A local GPIO write removes the USB serial link from that figure, but not the Pi's HDMI/display path, which is the more likely explanation for a lag of that size. Expect the GPIO change to tighten the *spread*; do not expect it to close the *lag*. Distinguishing the two requires the raster-position and display-path measurements, not a different trigger.

Related tests

- `tests/test_gv_sync` — `.gv` video playback: does `PlayGvFunc` put a frame on screen when it says it does?
- `tests/test_linuxgpio` — GPIO character-device output/input, used to verify header wiring before a session.
- `tests/test_parallel_port` — LPT data lines via `ppdev`, the equivalent check for a parallel-port trigger.
- `tests/test_dlpio8` — square-wave output for characterising the trigger box itself against an oscilloscope.
- `tests/test_clear_only_frames` — regression test for the compositor presentation bug described at the top of this document.
- `tests/test_vsync_blocking` — does `SDL_RenderPresent` block on VSYNC on this platform, or return immediately?

DLP-IO8

Driver: `triggers/dlpio8.go`. Full protocol notes and raw data at <https://github.com/chrplr/dlp-io8-g>; the timing figures below were measured there with a Siglent SDS1104X-E and are repeated because they change how you should use the device.

Lower the FTDI latency timer before reading anything

The DLP-IO8 is an FTDI device, and the `ftdi_sio` driver defaults to a **16 ms latency timer**: the chip holds a partly-filled buffer that long before sending it to the host. A poll the module answers instantly still takes 16 ms to come back.

Measured, $n=300$ per setting, for an 8-channel read:

latency_timer	round trip	poll rate
16 (default)	15.98 ms	63 Hz
4	3.99 ms	251 Hz
1	1.01 ms	995 Hz

The relationship is exactly $\text{round trip} = \text{latency_timer}$, so the module's own processing is negligible and the whole cost is driver batching. A polling loop gets the worst case rather than the average: waiting for each reply synchronises the loop to the timer and pays the full 16 ms every iteration.

```
echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

That reverts on replug. To make it stick:

```
# /etc/udev/rules.d/99-ftdi-latency.rules
SUBSYSTEM=="usb-serial", DRIVERS=="ftdi_sio", ATTR{latency_timer}="1"
```

The rule applies to every FTDI serial device on the machine, not just this one. That is usually what you want, but it is worth knowing it is system-wide.

It does nothing for sending triggers. Output latency is governed by USB frame scheduling. Do not expect this setting to make a trigger arrive sooner.

A multi-bit code is not atomic

There is no multi-channel command: every command is one ASCII byte affecting one line. `Send(mask)` therefore emits eight bytes which the module acts on as they arrive, and **the port takes ~610 μ s to settle**, showing partly-updated values throughout. Measured $n=99$: 86.2 μ s per byte, 609.5 μ s from the first line to the eighth.

Against a system sampling at 1 kHz that is about 61 % of a sample period, so a code change is sampled mid-transition roughly three times in five and recorded as a value that was never sent.

Use one line per event type, pulsed. A single line is one command byte, so there is no skew at all, and eight lines still distinguish eight event types. Reserve Send for a multi-bit code for the cases where the acquisition reads the code milliseconds after the onset edge rather than latching it at the edge, or where a strobe line is raised last once the code has settled.

Pulse width is only as good as your scheduling

The device has no pulse timer, so a pulse is two host writes and the width inherits host scheduling in full. Measured n=50 per width:

host state	median error	spread
idle	-10 to -20 μ s	$\leq 120 \mu$ s
under CPU load	up to +1.85 ms	up to 4.75 ms

The host's own busy-wait interval degrades by the same amount, tracking the wire to within 80 μ s — so the cause is the scheduler descheduling the process, not the USB path or the device. See [Setting priority under Linux](#); that is the fix, and it is a different mechanism from the latency timer above.

Parallel port and GPIO alternatives

Both avoid the USB path entirely: a write is one ioctl, not a USB serial transaction subject to frame scheduling. On hardware that offers either, this is the cheapest available improvement to onset-vs-TTL precision.

In the timing tests, select them with -trigger-device (see [Choosing a TTL output device](#)):

```
Timing-Tests -test av -trigger-device parallel
Timing-Tests -test av -trigger-device gpio -gpio-chip /dev/gpiochip0
```

or, for a whole session, TRIGGER_DEVICE=parallel / TRIGGER_DEVICE=gpio with run-timing-tests.sh.

In your own experiment code:

```
// Parallel port (LPT), 5 V. Needs `sudo modprobe ppdev` and the `lp` group.
p := triggers.NewParallelPort("/dev/parport0")
if err := p.Open(); err != nil { log.Fatal(err) }
defer p.Close()
p.Send(0x01) // all 8 data lines at once, D0 = DB25 pin 2

// GPIO character device (Raspberry Pi, Rock Pi, ...), 3.3 V. Needs kernel >= 5.10
// and the `gpio` group. Check which chip owns the header with `gpiodetect`.
g, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOChip("/dev/gpiochip0"),
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
)
```

```
if err != nil { log.Fatal(err) }  
defer g.Close()  
g.Pulse(0, 5*time.Millisecond)    // line 0 = the first pin in the array = BCM 17
```

Send(mask) sets all 8 lines simultaneously on both — unlike the DLP-IO8, where a multi-bit code is written one byte per line and takes $\sim 610\ \mu\text{s}$ to settle.

Note the voltage difference: parallel is 5 V, GPIO is 3.3 V. Confirm your acquisition system latches at 3.3 V before relying on the GPIO path.